

Pada bagian ini, dibahas secara rinci proses pemodelan deteksi kebangkrutan perusahaan asuransi menggunakan pendekatan Neural Networks. Tahapan pemodelan dilakukan secara sistematis untuk memperoleh hasil yang optimal, dimulai dari pengumpulan dan pra-pemrosesan data, pembentukan arsitektur jaringan saraf tiruan, hingga evaluasi performa model menggunakan metrik pengukuran yang relevan. Setiap tahapan dirancang untuk memastikan bahwa model mampu mengenali pola-pola tersembunyi dalam data keuangan perusahaan asuransi sehingga dapat memprediksi potensi kebangkrutan secara akurat. Dengan demikian, pembahasan berikut akan menjelaskan secara bertahap seluruh proses yang dilakukan, mulai dari tahap persiapan data hingga analisis hasil akhir model.

Tahapan Percobaan

1. Data Preprocessing

Sebelum dilakukan pemodelan, data terlebih dahulu melalui tahap *preprocessing* agar siap digunakan oleh algoritma klasifikasi. Langkah-langkah yang dilakukan adalah sebagai berikut:

a. Labeling Target

Variabel target yang semula berbentuk kategori teks diubah menjadi numerik menggunakan LabelEncoder.

```
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
y = le.fit_transform(y)
```

b. Seleksi Atribut

Atribut yang dianggap kurang relevan dihapus dari dataset.

```
data =
data.drop(columns=['No', 'Nama_Perusahaan', 'Z_Score', 'Z_Score_X1',
'Z_Score_X2', 'Z_Score_X3', 'Z_Score_X4', 'Tahun'])
```

c. Splitting Dataset

Dataset dibagi menjadi train (80%), test (10%), dan validation (10%) dengan stratified sampling untuk menjaga proporsi kelas.

```
from sklearn.model_selection import train_test_split
X = df.drop(columns=['Label'])
y = df['Label']

# train-test split
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

# validation split
X_test, X_val, y_test, y_val = train_test_split(
    X_temp, y_temp, test_size=0.5, stratify=y_temp,
    random_state=42
)
```

d. Feature Scaling

Berdasarkan distribusi data hasil *Exploratory Data Analysis (EDA)*, dilakukan dua jenis scaling:

- StandardScaler untuk fitur dengan distribusi mendekati normal.

```
from sklearn.preprocessing import StandardScaler, RobustScaler

scaler_std = StandardScaler()
scaler_rob = RobustScaler()
# Kolom
cols_standard = ['DAR', 'Komisaris_Independen', 'BoC_Size',
'TMT_Size_DK', 'TMT_Size_DD', 'Total_Asset']

# Fit hanya di training data
X_train_standard_scaled = pd.DataFrame(
    scaler_standard.fit_transform(X_train[cols_standard]),
    columns=cols_standard,
    index=X_train.index
)
```

- RobustScaler untuk fitur dengan distribusi tidak normal atau banyak outlier

```
# Fit hanya di training data
X_train_robust_scaled = pd.DataFrame(
    scaler_robust.fit_transform(X_train[cols_robust]),
    columns=cols_robust,
    index=X_train.index
)
```

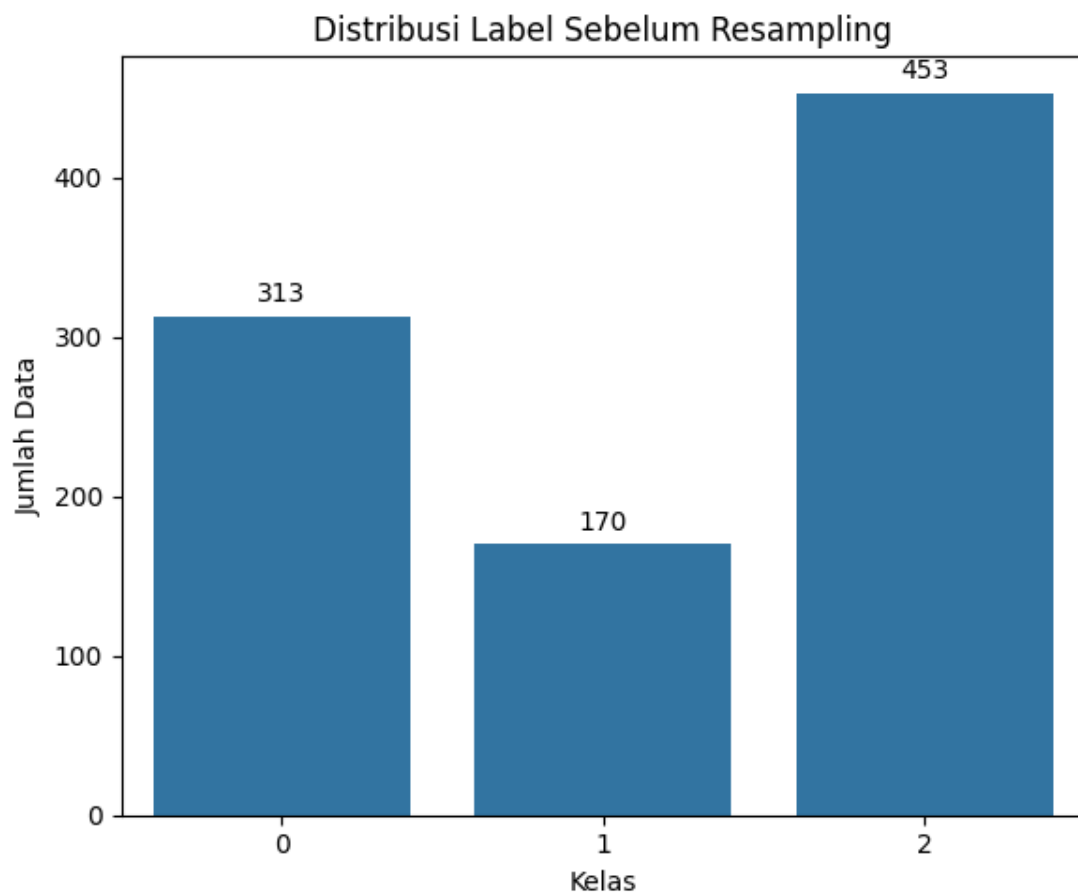
e. Penanganan Imbalance

Untuk mengatasi ketidakseimbangan kelas, digunakan teknik SMOTETomek, yaitu kombinasi *SMOTE* (oversampling kelas minoritas).

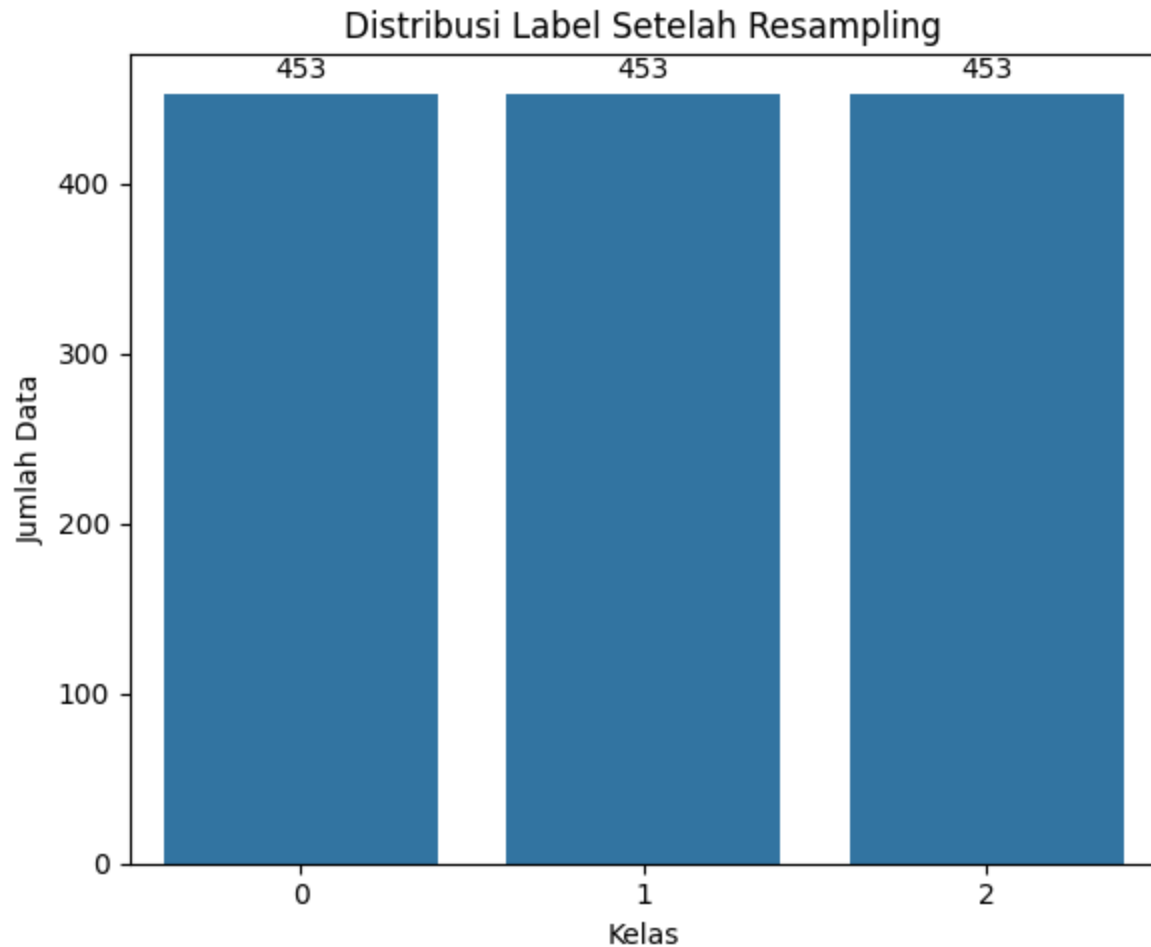
```
from imblearn.combine import SMOTE
smote = SMOTE(random_state= RANDOM_STATE)
X_train_res, y_train_res = smote.fit_resample(X_train,y_train)
```

Berikut perbandingan jumlah data per kelas antara sebelum dan sesudah dilakukan resampling menggunakan SMOTE

Sebelum Resampling



Berdasarkan grafik yang ditampilkan diatas, didapati bahwasannya data yang dimiliki ini lebih banuk didominasi oleh kelas 2 atau perusahaan sehat dibandingkan dengan kelas yang lainnya. Untuk mengatasi hasl tersebut dilakukan resampling menggunakan SMOTE dengan hasilnya sebagai berikut:



Berdasarkan grafik hasil resampling, untuk tiap tiap kelas yang kurang dari kelas mayoritas akan dilakukan resampling untuk bisa menyamai jumlah data di kelas mayoritas yaitu sejumlah 453 dengan harapan bahwa model bisa lebih general dan data yang diberikan juga bisa lebih luas.

2. Training Models Artificial Neural Networks

Setelah baseline model dengan Multinomial Logistic Regression disiapkan, langkah berikutnya adalah membangun dan melatih model Neural Network (ANN) untuk klasifikasi multikelas. Arsitektur ANN ditentukan melalui proses hyperparameter tuning dalam dua tahap menggunakan Optuna. Tahap pertama bertujuan menemukan jumlah layer, unit, aktivasi, batch size, dan learning rate yang optimal, sedangkan tahap kedua fokus pada penyetelan regularizer, dropout, dan batch normalization untuk meningkatkan kemampuan generalisasi model. Berdasarkan hasil tuning, konfigurasi final ANN disajikan dalam tabel berikut:

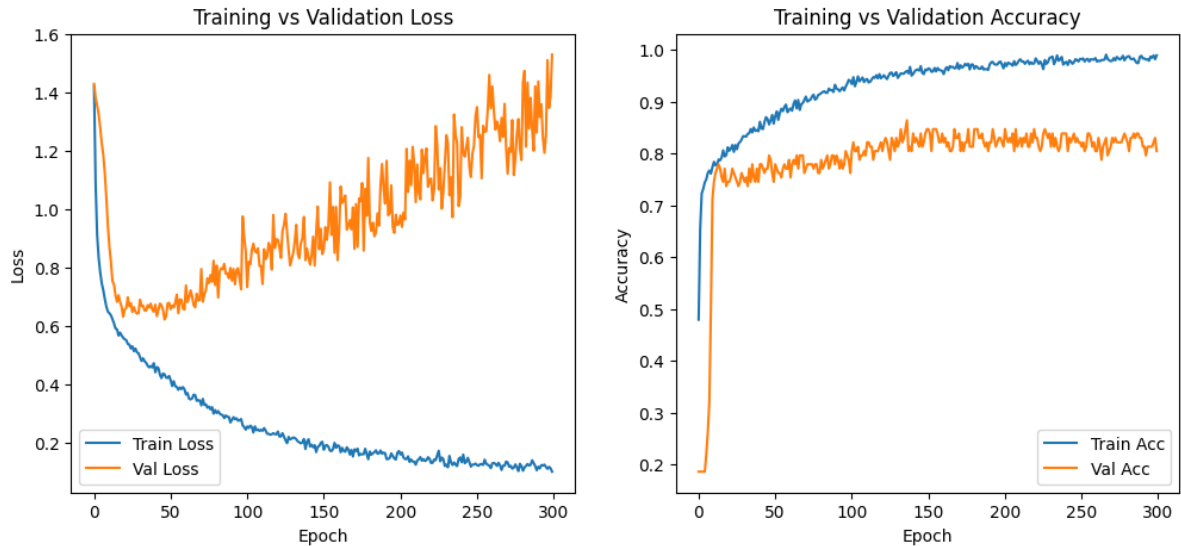
Layer	Units	Activation	Regularizer	Dropout	BatchNormalize
Input	18	-	-	-	-
1	128	Sigmoid	L2 (1.92e-05)	No	Yes
2	96	Sigmoid	L2 (3.24e-05)	0.131	No

3	96	Leaky ReLU	L2 (0.00381)	0.463	No
4	224	Softmax	L2 (9.21e-04)	No	Yes
Output	3	Softmax	-	-	-

Setelah melakukan dua tahap hyperparameter tuning menggunakan Optuna, konfigurasi final Neural Network (ANN) disusun seperti terlihat pada tabel di atas. Tabel ini menampilkan susunan layer mulai dari input, hidden layer, hingga output, beserta jumlah neuron, fungsi aktivasi, regularizer, dropout, dan batch normalization di setiap layer. Layer pertama memiliki 128 neuron dengan fungsi aktivasi sigmoid, dilengkapi L2 regularizer kecil dan batch normalization untuk membantu stabilisasi proses training. Layer kedua menurun menjadi 96 neuron dengan aktivasi sigmoid, diterapkan dropout sebesar 13,1% untuk mencegah overfitting. Layer ketiga juga memiliki 96 neuron dengan aktivasi Leaky ReLU dan dropout lebih besar yaitu 46,3%, untuk lebih meningkatkan generalisasi model pada pola data yang kompleks. Layer keempat menggunakan 224 neuron dengan aktivasi softmax, tetap menerapkan L2 regularizer dan batch normalization, untuk menyiapkan output yang lebih stabil sebelum menuju layer output. Layer output menggunakan 3 neuron dengan fungsi aktivasi softmax, menyesuaikan jumlah kelas pada dataset deteksi kebangkrutan perusahaan asuransi. Secara keseluruhan, konfigurasi ANN ini mencerminkan hasil tuning dua tahap: tahap pertama menentukan arsitektur inti (jumlah layer, neuron, aktivasi, learning rate, batch size), sedangkan tahap kedua menyempurnakan regularisasi, dropout, dan batch normalization. Susunan ini diharapkan mampu menangkap pola non-linear dan interaksi kompleks antarfitur, sehingga memberikan performa yang lebih baik dibandingkan model linear sederhana seperti Multinomial Logistic Regression.

3. Evaluation Model Neural Networks

Untuk menilai kinerja model *Neural Network* yang telah dibangun, dilakukan proses evaluasi berdasarkan hasil pelatihan serta performa pada data validasi dan data uji. Hasil evaluasi ini ditampilkan dalam bentuk grafik yang memvisualisasikan perkembangan *loss* dan akurasi selama proses training, serta dilengkapi dengan *classification report* dan *confusion matrix* guna memberikan gambaran yang lebih detail terkait kemampuan model dalam melakukan klasifikasi.



Proses pelatihan model *Neural Network* divisualisasikan pada diatas, yang memperlihatkan kurva *loss* dan akurasi pada data latih serta validasi. Dari grafik terlihat bahwa *training loss* terus menurun secara konsisten, sementara *validation loss* justru mulai meningkat setelah sekitar epoch ke-50. Hal ini mengindikasikan bahwa model mulai mengalami *overfitting* pada tahap tersebut. Fenomena serupa juga tampak pada grafik akurasi, di mana akurasi pada data latih terus meningkat hingga mendekati 100%, sedangkan akurasi validasi cenderung stagnan pada kisaran 80–82% setelah epoch ke-100. Hal tersebut sudah dapat diantisipasi agar model yang didapatkan Adalah model dengan hasil terbaik, pada saat training ditambahkan sebuah parameter Callbacks untuk mengembalikan bobot bobot dengan hasil terbaik pada model saat ditraining. Pada gambar dibawah ini akan ditampilkan untuk bagaimana performa model pada saat dilakukan evaluasi terhadap data testing dan data validation.

Classification Report:

	precision	recall	f1-score	support
Area abu-abu	0.55	0.57	0.56	21
Perusahaan Sehat	0.89	0.89	0.89	57
Potensi Bangkrut	0.84	0.82	0.83	39
accuracy			0.81	117
macro avg	0.76	0.76	0.76	117
weighted avg	0.81	0.81	0.81	117

Kelas Area abu-abu -> Precision: 0.5455, Recall: 0.5714, F1-score: 0.5581

Kelas Perusahaan Sehat -> Precision: 0.8947, Recall: 0.8947, F1-score: 0.8947

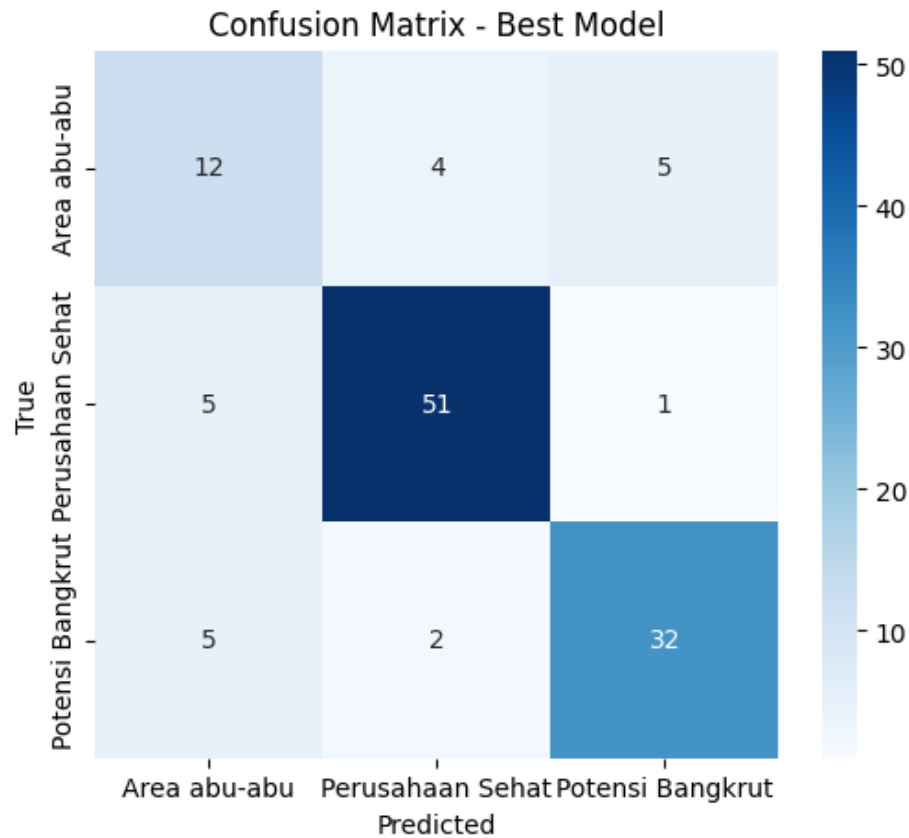
Kelas Potensi Bangkrut -> Precision: 0.8421, Recall: 0.8205, F1-score: 0.8312

Macro Avg -> Precision: 0.7608, Recall: 0.7622, F1-score: 0.7613

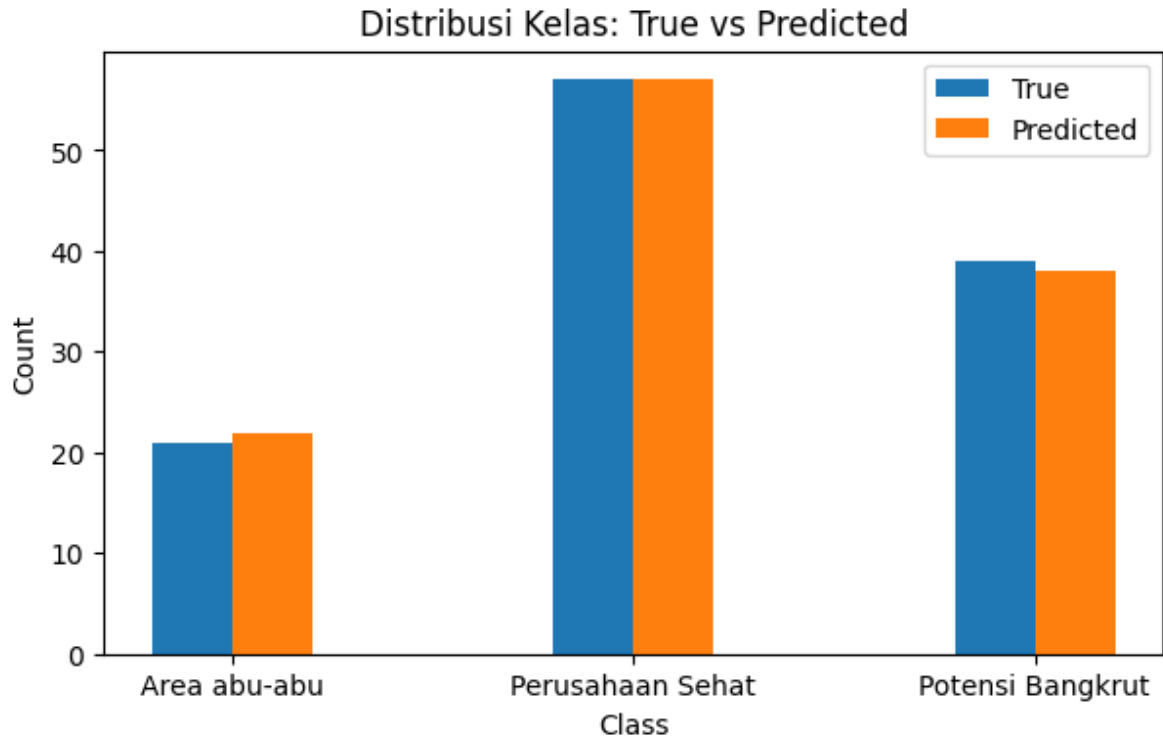
Weighted Avg -> Precision: 0.8145, Recall: 0.8120, F1-score: 0.8131

Evaluasi model pada data uji menghasilkan akurasi sebesar 81%, dengan skor rata-rata (*macro average*) precision, recall, dan F1-score masing-masing sebesar 0.76, 0.76, dan

0.76. Jika dilihat per kelas, performa model berbeda-beda. Kelas 1 memperoleh hasil terbaik dengan precision, recall, dan F1-score sebesar 0.89. Kelas 2 juga menunjukkan performa cukup baik dengan F1-score 0.83, sedangkan kelas 0 memiliki hasil paling rendah dengan F1-score 0.56. Hal ini menunjukkan bahwa model lebih mampu mengenali pola pada kelas mayoritas (kelas 1 dan kelas 2) dibandingkan kelas minoritas (kelas 0).



Berdasarkan confusion matrix, dapat dilihat bahwa performa model bervariasi pada tiap kelas. Untuk kelas 0, dari 21 sampel yang ada, sebanyak 12 sampel berhasil diprediksi dengan benar, sementara 4 sampel salah diklasifikasikan sebagai kelas 1 dan 5 sampel lainnya salah diprediksi sebagai kelas 2. Hal ini menunjukkan bahwa sekitar 42% data kelas 0 masih mengalami salah klasifikasi, sehingga model cukup lemah dalam mengenali kelas ini. Pada kelas 1, performa model terlihat paling unggul, di mana dari 57 sampel, sebanyak 51 sampel berhasil diprediksi dengan benar, hanya 5 sampel salah ke kelas 0, dan 1 sampel salah ke kelas 2. Tingkat kesalahan yang rendah ini mengindikasikan bahwa model sangat andal dalam mengklasifikasikan kelas 1. Adapun pada kelas 2, dari total 39 sampel, sebanyak 32 diprediksi dengan benar, sedangkan 5 sampel salah diklasifikasikan ke kelas 0 dan 2 sampel salah ke kelas 1. Meskipun masih terdapat kesalahan, performa model pada kelas ini tetap tergolong baik. Secara keseluruhan, model menunjukkan performa terbaik pada kelas 1, cukup baik pada kelas 2, namun masih kurang andal dalam mengklasifikasikan kelas 0.



Berdasarkan grafik perbandingan precision, recall, dan F1-score per kelas, terlihat bahwa performa model bervariasi pada setiap kelas. Pada kelas 0, nilai precision, recall, dan F1-score relatif rendah, yaitu berkisar antara 0,55 hingga 0,57. Hal ini menunjukkan bahwa model masih sering melakukan kesalahan dalam mengenali kelas tersebut, baik dalam bentuk *false positives* maupun *false negatives*. Sementara itu, pada kelas 1 model menunjukkan performa terbaik dengan nilai precision, recall, dan F1-score yang tinggi dan seimbang, yaitu sekitar 0,90. Kondisi ini mengindikasikan bahwa model mampu mengklasifikasikan kelas 1 dengan sangat baik dan konsisten. Adapun pada kelas 2, nilai evaluasi juga cukup tinggi, berkisar antara 0,82 hingga 0,85, meskipun sedikit lebih rendah dibandingkan kelas 1. Hal ini menunjukkan bahwa model masih cukup andal dalam mengenali kelas 2, meskipun terdapat beberapa kesalahan prediksi. Secara keseluruhan, dapat disimpulkan bahwa model bekerja optimal pada kelas 1, cukup baik pada kelas 2, namun masih lemah dalam mengklasifikasikan kelas 0.

Secara keseluruhan, model Neural Network yang dibangun menunjukkan performa yang cukup baik dengan tingkat akurasi keseluruhan sebesar 81%. Hasil evaluasi memperlihatkan bahwa model mampu mengenali kelas 1 dengan sangat baik dan konsisten, sedangkan performa pada kelas 2 juga tergolong cukup tinggi meskipun masih terdapat beberapa kesalahan prediksi. Namun, pada kelas 0 model masih kurang andal, ditunjukkan oleh nilai precision, recall, dan F1-score yang relatif rendah serta tingginya tingkat mis-klasifikasi ke kelas lain. Selain itu, pola pada grafik *training vs validation* mengindikasikan adanya potensi *overfitting* setelah melewati sejumlah epoch, di mana akurasi training terus meningkat sementara akurasi validasi cenderung stagnan dan loss validasi meningkat. Dengan demikian, meskipun model telah menunjukkan hasil yang

cukup baik secara keseluruhan, terdapat peluang untuk peningkatan, khususnya dalam mengatasi *overfitting* dan memperbaiki kemampuan klasifikasi pada kelas 0.